Polaris Flow 维护者速查卡

v1.0 · 2026-10-19 · 受众：工作流维护者 · 配套：速查卡.html（团队推广版，勿混用）

① 三层架构与四条铁则 入口 → 调度 → 执行

入口层 tweak / normal / strict

薄编排技能，**只声明档位**，不代做任务判定与阶段衔接。命令层（/psfl:*）同样只做「选择 + 路由」，不产任何产物。

调度层 intake

执行层**唯一入口**。Step 1 恢复分发（在途优先）→ Step 2 入口粗判档 → Step 3 任务初始化（仅零在途）→ Step 4 转交。六出口：`resume / handover / create / select / resolve-drafts / block`。

执行层 阶段技能

specify → plan → design(可选) → tasks → build → verify → ship。阶段技能**不得互相调用、不得直接调 intake**；出口经 `state next` 推进游标。retro 是调度层唯一豁免：由用户直接触发，不需 task_id。

铁则：**档位升降唯一位置 = specify 出口**，过即冻结，其余阶段纯按 skill 字段转交；intake 复核只升不降；上游已给 task_id 时跳过枚举（机器交接免提问）。

② 档位与值域 写错一个字母，面板显示「未知阶段」

字段	合法值	说明
`workflow.mode`	`tweak | normal | strict`	中文名「工作流」；**full 已退役**（与 verify_mode 的 full 同名不同义），存量 full/delivery 不再识别
`complexity_level` `risk_level / current_tier`	`trivial | standard | critical`	方向 critical > standard > trivial；分级→档位映射单源在 risk-signals.md §1，技能侧不内联
`phase`	`specify | plan | design | tasks | build | verify | ship | retro`	写入前用 `writablePhaseList` 自查（含 idle）；空串/idle 合法，写错现在什么都不报，几个月后面板才暴露

入口预声明：C01 → trivial → tweak · C02 → standard → normal · C03 → critical → strict　权威判定仍在 specify 出口

③ 引用与命名红线 踩线即返工

类别	规则
技能引用	`polaris:coding:verify`（**无斜杠**）；命令引用 `/psfl:coding:tweak`（带斜杠）；**「技能名加斜杠」是坏引用**
菜单编号	C01 / M01 / R02 / P01 / T01 只属于**命令层**；技能内指路用技能名，禁止反向引用命令或菜单编号；档位用 tweak / normal / strict 表述
state 写入	一律 `scripts/task-state-entry.sh`，**禁止直写 yaml 块**；字段缺失是常态（draft-create 只写 kind: coding，骨架不落盘），靠增量长出
候选呈现	`<task_id>（工作流：<mode_label>，阶段：<phase_label>，技能：<skill_label>）` 单行格式
双语言	先写 `assets/zh/`，用户确认后再同步英文；**未同步前不写 Changelog**；改技能后测试全绿方可提交
配置归属	配置归代码，**禁止用 policy 承载配置表**；不保留向后兼容，删旧路径不加兼容层

权威源：src/core/config/task-kind-layout.ts（阶段/产物/档位唯一真相）· assets/zh/commands/（命令）· assets/zh/skills/（技能）

④ 阶段 → 产物 → 归档矩阵 按 kind 五条链，落盘前先查这张表

类型	阶段（技能）	产物
coding 开发	specify 澄清	.polaris/tasks/<id>/intention.md（可迁 openspec/changes/<id>/）
	plan 提案	openspec/changes/<id>/{proposal.md, design.md, specs/}
	design 深度设计*	openspec/changes/<id>/detailed-design.md（仅 strict；plan 出口询问是否深化）
	tasks 任务规划	openspec/changes/<id>/tasks.md（复选框；set-checkbox 与 runtime.build 计数原子同步）
	build 构建	代码与回归测试（缺陷修复必含 TDD 标记的回归用例）
	verify 验收	openspec/changes/<id>/reviews/verify-report.md
	ship 交付	归档 .polaris/archive/<id>/
requirement 需求 PRD	discovery 澄清	req_baseline.md + req_clarify_summary.md
	draft 草稿	prd-draft.md + _baseline_index.json + _key_points.json
	refine 完善	prd-final-v1.0.md（内派 review / testability 双评审）
	ship 交付	归档 docs/prd/；R03 readiness FAIL 阻断并回退 refine
testcase 测试用例	discovery 澄清	.polaris/testcases/<id>/testcase_plan.md
	draft 草稿	test-cases.md（来源严格限定为定稿 PRD）
	refine 完善	test-cases-{nonfunctional,interface,automation}.md + testcase-review-report.md（review Critical 清零才放行）
	ship 交付	testcase-report.md → 归档 docs/testcases/<id>/
prototype 原型	blueprint 蓝图	blueprint.md + task-card.md（人工确认后才建造）
	build 构建	高保真 HTML 原型（文件名由蓝图定，无固定名，故不登记）
	ship 交付	归档 docs/prototype/；review 是服务型技能，游标不落
debug 缺陷修复	diagnose 诊断与方案	diagnose-brief.md + reviews/rca-report.md + tasks.md（通道 hotfix / bugfix 由场景分流步判定）
	patch 实现与自验	verification.md（hotfix 通道含五维独立验证）
	closeout 关单	reviews/bugfix-report.md；归档 docs/troubleshooting/<id>/——**不用 .polaris/archive、不走 openspec、不交 ship**

* design 为可选阶段：UI 规则是「已越过渲染为已跳过，而非已完成」。任务目录根：`.polaris/tasks/<id>/`（testcase 族在 `.polaris/testcases/<id>/`）；requirement / prototype / debug 不建 draft 目录，直接正式 task_id。

⑤ state.yaml 写入纪律 task-state-entry.sh 是唯一写入口

写什么

- 阶段推进：`complete-phase`，`--next-phase` 可省——缺省由 `nextPhaseForMode` 按档位派生（plan→design/tasks 二分支仍显式传）
- 游标语义：出口把游标设成「接下来要执行的阶段」，**phase 即待执行阶段**，与技能同名映射
- 勾选同步：`set-checkbox` 同锁内写 tasks.md + runtime.build 计数

别写什么

- 禁止直写 yaml 块；**worktree 块的 appendFile 是全仓唯一绕过处**，重复顶层键会让 yaml 库直接抛错、该文件此后全挂
- `runtime.propose.*` 是孤儿段——propose 已改名 plan，正确路径 `runtime.plan.*`
- `runtime.build.pause_reason` 等技能自造字段未登记类型，用前先查 state.example.yaml

⑥ CLI 速查 polaris --help 仅列这些；hooks 运行时命令走 polaris-flow

命令	用途 / 要点
`polaris init`	初始化工作流 + Harness 环境；Superpowers 依赖 GitHub——不通时用 `POLARIS_GITHUB_MIRROR` 镜像或 `POLARIS_SUPERPOWERS_PATH` 本地目录
`polaris status`	当前 change 与工作流状态
`polaris dashboard`	本地工作台，API 与前端同端口；`--api-only` 供前端 HMR
`polaris doctor`	诊断环境、schema 与 skill 安装状态
`polaris update`	三层范围可单选：program（本体）/ assets（skills·commands·agents·rules·hooks，按指纹跳过）/ deps（openspec·superpowers·codegraph）。选项 `--only --skip --force --prune --lang --scope --json`；例：`update --only skills,commands`
`polaris uninstall`	卸载组件（占位，尚未实现）